Database Design and Implementation Report

HOTEL RESERVATION SYSTEM DATABASE

Module: Databases and Big Data
Student Name: Divakar Sabarwal
Student ID: GH1047352
Submitted To: Mr. Alireza Mehmoudi

GIThub Repository:
https://github.com/sabarwaldivakar/Divakar-s-Hotel-Reservation-System/tree/main

Youtube Video: https://youtu.be/qLgZ7p9H_1U

TABLE OF CONTENTS

INTRODUCTION

In the modern Digital world, everybody wants to keep their data digitally and decrease keeping storage rooms for the past books of records. So, in the same way hotels, e-commerce businesses, HealthCare sector and every other entity want to keep their records online. There comes the Databases to help with these data storing problems and allow users to store data in a systematic and Digital Manner.

So, for the project I chose the topic to make the database for a hotel reservation system. The aim was to design and implement a relational database using MariaDB. It was developed to help with the main hotel operations like customer handling, bookings, room management, payments tracking, and services.

The project required analysis, database normalisation, tables and relationships, data population and query development. An Entity Relationship Diagram (ERD) was created to make an overview of how the database would look and what all relationships were needed.

The final Database is a fully functional database for managing a Hotel Reservation System which demonstrates the designing concepts of database such as the primary key, foreign key, normalization, and others.

Project Overview

The Hotel Reservation System database was developed to manage the main operations of a hotel in an organised and efficient manner. I tried to enable efficient management of customer records, bookings, rooms, payments and services. By centralizing this information within a relational database, the hotel can now maintain accurate records, reduce data redundancy, and improve operational efficiency.

The database is made of seven interconnected tables. The Customers table stores information about customer such as names, contact details, and addresses.

The Room Types table contains information about different categories of rooms, including their descriptions, prices, and capacities.

The Rooms table stores details of individual rooms from the hotel.

The Bookings table records reservation details which further links it to customers with the rooms they have booked.

The Payments table has the details of the payments made for each booking and helps the hotel track the revenue generated from the bookings.

The Services table contains information about other services the hotel offers such as breakfast, spa access, gym access, massage services and airport transfer.

To support the relationship between the bookings and services, a junction table called Booking Services was implemented. This table allows multiple bookings to have multiple services and multiple services to be in multiple bookings.

The structure provides a solution for the management of hotel reservations, ensuring the consistency of the data by means of primary keys, foreign keys, constraints and relational design principles.

3. Database Design

3.1 Entity Relationship Diagram (ERD)

The database structure was initially designed by making an Entity Relationship Diagram (ERD) which presented the entities, attributes, and relationships within the hotel reservation system virtually. The ERD provides a blueprint for the database and made it easy to build.

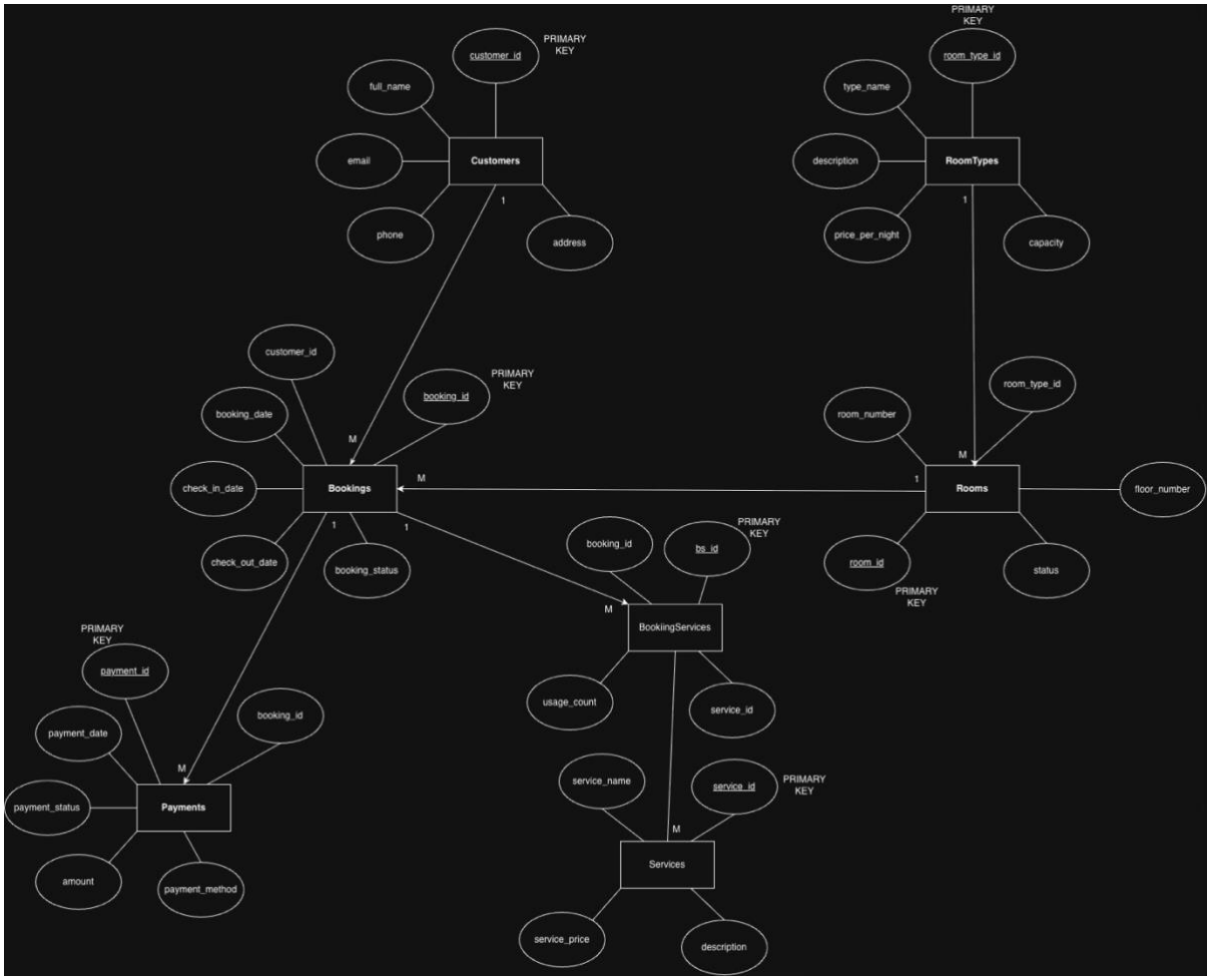


Figure 1: Entity Relationship Diagram of the Hotel Reservation System

The ERD has seven entities: Customers, Room-Types, Rooms, Bookings, Payments, Services, and Booking-Services. Each entity has a primary key which uniquely identifies records. Relationships between tables are established through Foreign Keys.

3.2 Relationships

The Database was made with many one-to-many relationships. A customer can have multiple bookings, but each booking belongs to only one customer. Just like that, a room type can be associated with multiple rooms, but each room belongs to a single room type.

Rooms and bookings are also connected with a one-to-many relationship, which allows the same room to be booked multiple times at different time periods.

The payment information from each booking is stored in the Payments table which is also linked to Bookings table.

Bookings and Services have a many-to-many relationship because a booking can use multiple services similarly, services can be used by multiple bookings. Then I created a junction table called BookingServices to maintain this many-to-any relationship.

3.3 Normalization

The normalization of the database done to Third Normal Form (3NF) so that data redundancy can be minimised, and data integrity can be maintained.

To achieve the First Normal Form (1NF) it was ensured that each table contains atomic values and no repeated groups exist.
For the Second Normal Form (2NF) all non-key attributes were made fully dependent on their respective primary keys.
By removing transitive dependencies and ensuring that non-key attributes depend only on the primary key of their own table Third Normal Form (3NF) was achieved.

With the help of Normalization the consistency is improved, data repetition is reduced and made it easier for future maintenance of the database.

4. Database Implementation

4.1 Tables and Relationships

MariaDB was used for the creation of the Hotel Reservation System database. To store and manage hotel-related information a total of seven tables were created. The tables are Customers, Room-Types, Rooms, Bookings, Payments, Services, and Booking-Services.

Each Table has its own ‘Primary key’ to uniquely identify records. To establish relationships between tables ‘Foreign key’ constraints were used. For example, the Bookings table references both Customers and Rooms, and the Payments table references Bookings.
Bookings-Services is the Junction table used to satisfy the many-to-many relationship between Bookings and Services.

4.2 Constraints

To improve data integrity and prevent invalid data entry multiple constraints were added to the database.

Use of Primary key to ensure uniqueness of records in each table. Foreign key constraints were implemented for maintaining relationships between tables and preventing orphan records. Unique constraints were applied to attributes such as customer email addresses, room numbers, service names, and room type names so that there are no repeated values in these columns and makes it easy to distinguish the customers.

Where appropriated Default values were also used. For example, newly created rooms are automatically assigned an “Available” status, while bookings and payments receive default statuses of “Confirmed” and “Completed” respectively.

To make sure that the check-out date is always later than the check-in date a CHECK constraint was added within the Bookings table.

4.3 Sample Data

Inserted sample data into all tables to check the functioning of database with running the realistic hotel operations. The database contains multiple room types, hotel rooms, customers, bookings, payments, and additional services. To showcase reservation activity and revenue generation sample booking records were also created across several months.

The Booking-Services table was populated with service usage records. Allowing the customers with the access to additional facilities such as breakfast, gym access, spa services, massage treatments, airport shuttle, and pool access.

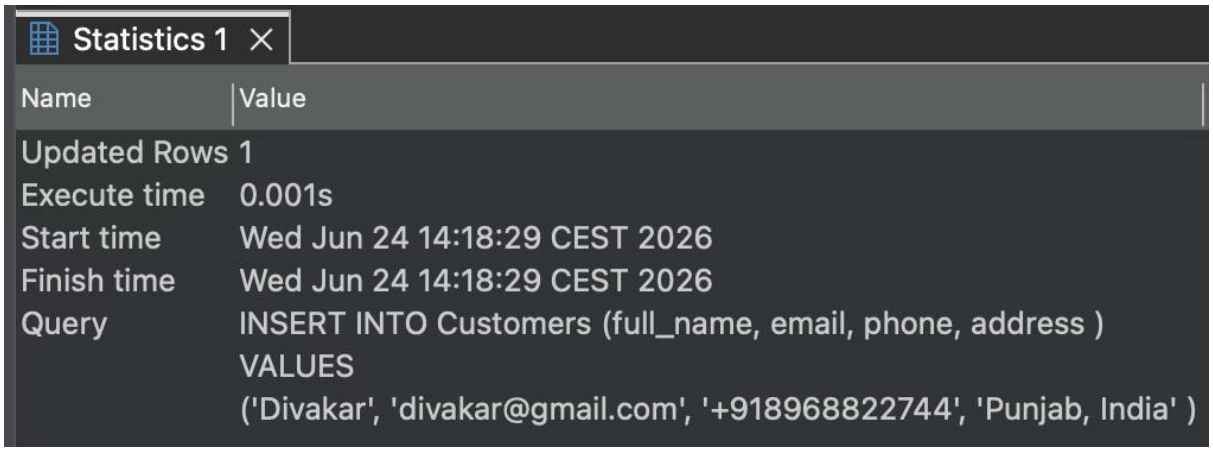
5. Query Implementation and Testing

To demonstrate the functionality of the Hotel Reservation System database several SQL Queries were developed and tested. These queries are used to manage records, retrieve booking information, analyse revenue, and support business decision-making.

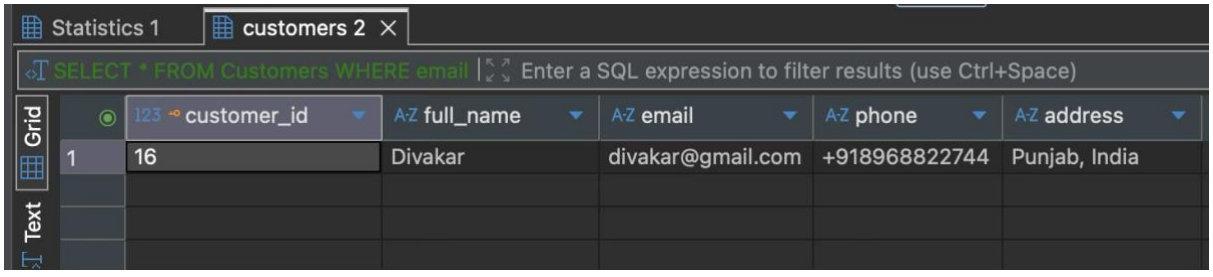
5.1 Customer Management

To create, retrieve, update, and delete records customer management queries were implemented.

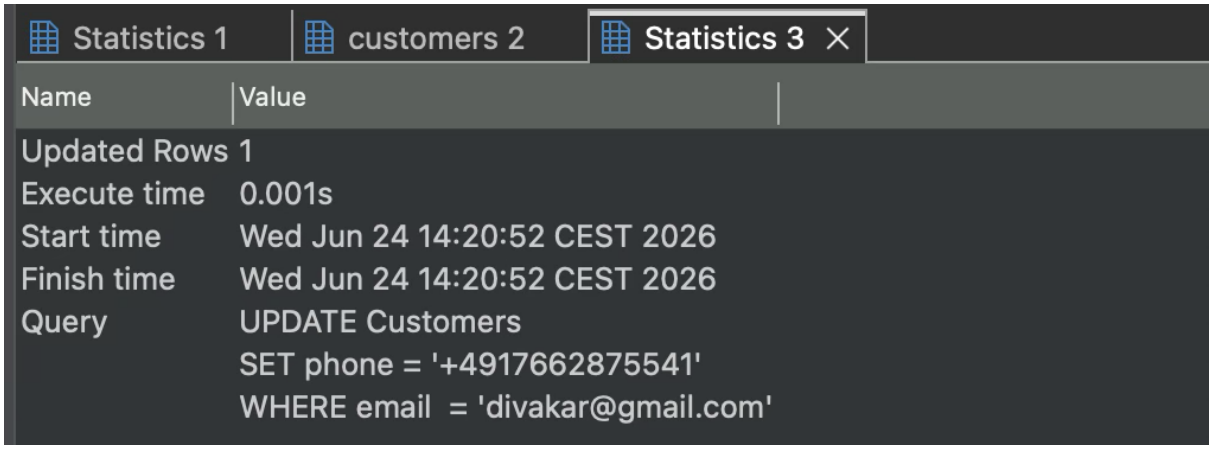
Query 1: Using an INSERT statement insert a new customer into customer's table.



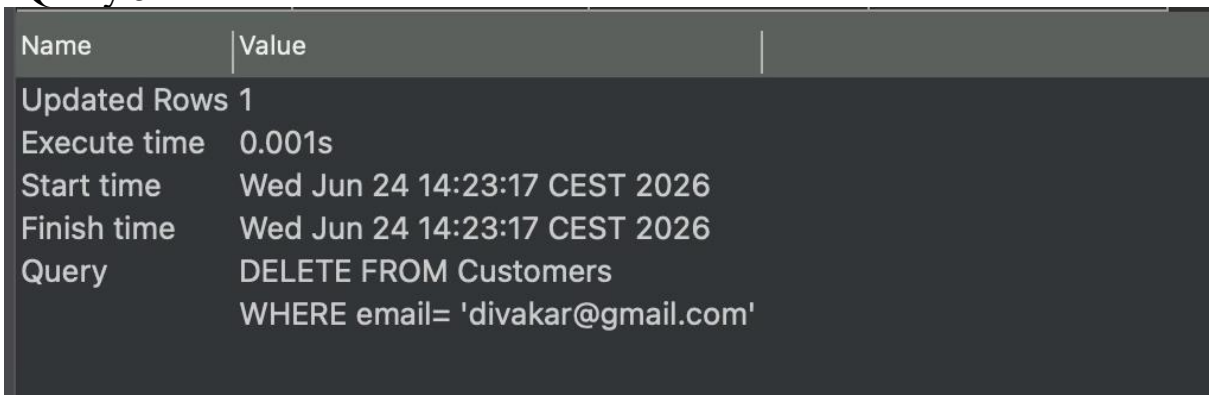
Query 2: Using a SELECT statement verify the inserted customer.



Query 3: In the customer's record update a customer's phone number.



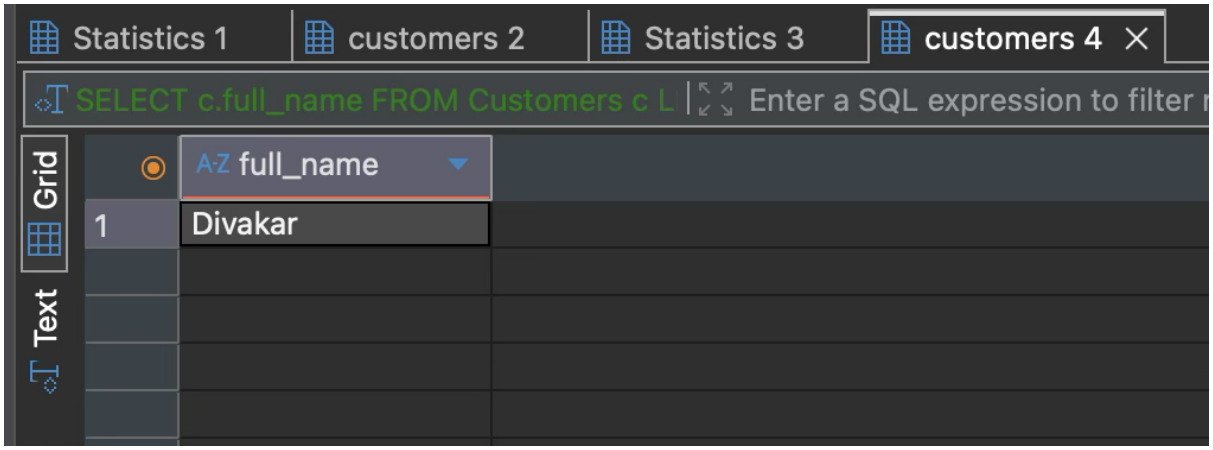
Query 5: Delete the customer record from the database after verification.



The complete lifecycle of customer record management within the system is demonstrated by the queries above.

5.2 Customer Booking Analysis

Query 4: To identify customers who do not currently have any bookings a Left join query was implemented.



5.3 Booking Information Retrieval

Query 6: To get the information about Bookings multiple Inner joins were used. Information from the Customers, Bookings, Rooms, and Room-Types tables was used to provide a complete overview of hotel reservations.

Customer names, room numbers, room categories, and booking dates were displayed in the output.

	booking_id	full_name	room_number	type_name	check_in_date	check_out_date
1	1	Max Müller	101	Standard	2026-03-01	2026-03-04
2	2	Anna Schmidt	201	Luxury	2026-03-05	2026-03-08
3	3	Lukas Weber	301	Family	2026-03-10	2026-03-14
4	4	Sophie Fischer	401	Suite	2026-03-15	2026-03-20
5	5	Leon Wagner	102	Standard	2026-03-22	2026-03-25
6	6	Emma Becker	202	Luxury	Value: 2026-03-22 Timezone: GMT+01:00	2026-04-05
7	7	Noah Hoffmann	302	Family		2026-04-12
8	8	Mia Schneider	402	Suite	2026-04-15	2026-04-20
9	9	James Wilson	103	Standard	2026-04-22	2026-04-25
10	10	Pierre Martin	203	Luxury	2026-04-28	2026-05-02
11	11	Marco Rossi	303	Family	2026-05-05	2026-05-09
12	12	Carlos Garcia	403	Suite	2026-05-12	2026-05-17
13	13	Priya Sharma	104	Standard	2026-05-20	2026-05-24
14	14	Li Wei	204	Luxury	2026-05-27	2026-05-31
15	15	Ahmed Hassan	404	Suite	2026-06-05	2026-06-10

5.4 Booking and Revenue Statistics

To analyse booking activity and revenue generation different aggregate queries were developed.

Query 7: Calculates the total number of bookings stored in the database using the COUNT () function.

	123 Total_Bookings
1	15

Query 8: Calculates revenue generated through different payment methods using the SUM () function and GROUP BY clause.

	A-Z payment_method	123 Total_Revenue_by_Payment_method
1	Cash	587.91
2	Credit Card	5,233.59
3	Debit Card	3,543.7
4	PayPal	2,616.77

Query 9: Calculates the average booking payment using the AVG () function.

	123 Average_Booking_Payment
1	798.798

To know about the operational performance and customer payment behaviour these queries can be used.

5.5 Service Revenue Analysis

Query 10: To calculate the revenue generated by each hotel service a service revenue query was developed. The query multiplies service prices by their corresponding usage counts and aggregates the results using the SUM () function.

	service_id	service_name	Total_service_revenue
1	7	Airport Shuttle	149.95
2	1	Breakfast	143.91
3	3	Dinner	209.94
4	4	Gym Access	39.96
5	2	Lunch	124.95
6	6	Massage	239.97
7	8	Pool Access	103.92
8	5	Spa Access	149.97

This information can be helpful for the management to identify the most profitable services offered by the hotel.

5.6 Room Revenue Analysis

Query 11: To calculate revenue generated from room bookings only a room revenue query was created. To determine the duration of each stay DATEDIFF () is used and multiplied by the room's nightly rate to give the total cost of room.

	booking_id	full_name	Total_Room_Revenue
1	1	Max Müller	269.97
2	5	Leon Wagner	269.97
3	9	James Wilson	269.97
4	13	Priya Sharma	359.96
5	2	Anna Schmidt	449.97
6	6	Emma Becker	599.96
7	10	Pierre Martin	599.96
8	14	Li Wei	599.96
9	3	Lukas Weber	799.96
10	7	Noah Hoffmann	799.96
11	11	Marco Rossi	799.96
12	4	Sophie Fischer	1,249.95
13	8	Mia Schneider	1,249.95
14	12	Carlos Garcia	1,249.95
15	15	Ahmed Hassan	1,249.95

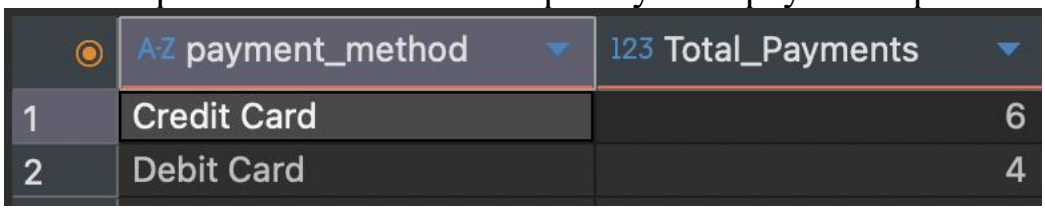
This gives an estimate of revenue generated by each booking.

5.7 Payment Method Analysis

Query 12: To identify payment methods that were used more than three times a HAVING query was used. Firstly, it groups payment records by

payment method and then the aggregate results are filtered using the HAVING clause.

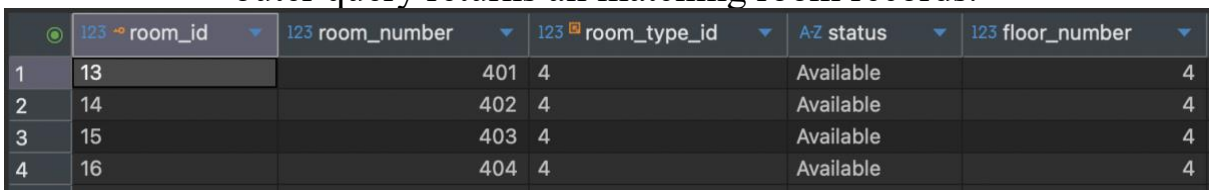
This helps to know the most frequently used payment options.



	payment_method	Total_Payments
1	Credit Card	6
2	Debit Card	4

5.8 Room Type Search

Query 13: A subquery to retrieve all rooms that belonged to Suite category was implemented. The inner query identifies the room type, while the outer query returns all matching room records.

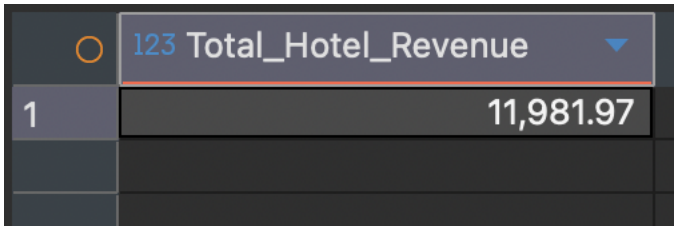


	room_id	room_number	room_type_id	status	floor_number
1	13	401	4	Available	4
2	14	402	4	Available	4
3	15	403	4	Available	4
4	16	404	4	Available	4

The use of nested queries for advanced data retrieval is demonstrated.

5.9 Total Hotel Revenue

Query 14: A final revenue query to calculate the total revenue generated by the hotel using the SUM () function on all payment records.



	Total_Hotel_Revenue
1	11,981.97

This query gives the hotel's financial performance and can be good for taking further.

6. Challenges and Solutions

The main challenge was to think of the entities and its attributes once it was done the biggest step towards the creation of the database.
Then came choosing the constraints for each table they were not so hard but yeah, they took a bit time.
The tables are now created comes the insertion so randomly put in a list of customers their bookings, amounts, services and every table was filled out.

Then came the main part to put in meaningful Queries so I looked into the Queries done in the class took some from them and then thought the database as my own business and worked through what I would need. Like calculating the revenues, looking for customers without bookings, looking for per room revenue.
The only problem that I found and could not actually solve was calculating total amount for a booking without getting duplicate rows for the same customer_id.

	123 customer_id	123 Total_Booking_Revenue
1	1	301.95
2	1	329.95
3	2	513.93
4	2	469.95
5	3	874.93
6	3	851.92
7	4	1,329.94
8	4	1,349.93
9	5	301.95
10	6	739.92
11	7	819.94
12	8	1,279.94
13	9	285.96
14	10	649.95
15	11	959.94
16	12	1,301.91
17	13	409.94
18	14	669.94
19	15	1,309.93

7. Conclusion

The design and implementation of the Hotel Reservation System was completed successfully. The system now provides a full verified method for managing customers, payments, bookings and hotel services through a structured Relational Database.

The Database is now fully capable of doing all the realistic hotel operations while maintaining data integrity. The Project has achieved its objectives and shows practical implementation of Database design and management concepts that were taught in the class.

References

Mehmoudi, A. (2025) Database and Big Data Lecture Slides. Gisma University of Applied Sciences.

MariaDB Foundation (2025) MariaDB Documentation.